

Chapter 5: System/Resource Analysis

5.1 Study of Current System Resources

At the time of developing the IMSAxes Automation project, the internal tool/utility intended to automate the execution of performance and functional scripts was not fully established. Additionally, the main web application under test was itself in an evolving development phase. This led to some constraints while framing reusable test modules and scenarios, as several UI components and workflows were still changing.

5.2 Current System Problem and Weakness

- **Instability in Unexpected Scenarios:** The existing framework exhibited significant vulnerability to abrupt environmental interruptions, such as power transients, unexpected network disconnections, or forced system shutdowns. Due to the lack of dynamic state-saving mechanisms and automated recovery protocols, any terminal failure during execution resulted in aborted operations, potential data corruption in test logs, and the necessity for complete manual restarts, thereby reducing overall system reliability.
- **Incomplete Automation Readiness:** The automation suite demonstrated a lack of resilience against the rapid deployment cycles of the base Windows application. Because the scripts relied heavily on static locators, any unannounced modifications to the underlying UI architecture inevitably led to broken test flows ("flaky tests"). This constant need for manual script refactoring and recalibration created continuous maintenance overhead, severely hindering the timeline for achieving complete "automation readiness."
- **Compatibility Limitations:** The integration architecture struggled with seamless cross-environment operability when managing WebDriver instances. Technical bottlenecks were frequently observed during complex session handling-particularly when context-switching between different test domains or attempting to maintain persistent sessions across varied browser engines. This incompatibility often resulted in synchronization artifacts, stale element references, and premature session timeouts.

5.3 Requirements of New System

5.3.1 Functional Requirements

- **Module 1: Automated Fault Tolerance and Recovery (Robust Recovery Handling)**
 - Built-in exception handling to detect Windows application crashes and driver connection drops.
 - Ability to automatically relaunch the application .exe and continue execution after an unexpected closure.

- Significant reduction in false test failures caused by OS-level interruptions like Windows pop-ups or focus losses.
- **Module 2: Orchestrated Execution Environment (Enhanced Execution Utility)**
 - Completely independent script execution on dedicated Windows virtual machines without manual triggers.
 - Accurate statistics on total Windows tests run, passed, failed, or skipped due to hard application hangs.
 - Support for batch execution of grouped test cases against specific Windows application versions.
 - Search filters to rapidly locate specific scripts or historical Windows desktop execution logs.
- **Module 3: Intelligent Telemetry and Logging Mechanism**
 - Detailed and structured logs capturing simulated mouse/keyboard inputs and Windows Control ID interactions.
 - Failure cause indication with dumps of the Windows UI Automation tree for trace-level debugging.
 - Execution telemetry accessible across teams centrally, avoiding the need for Remote Desktop (RDP) to test machines.
- **Module 4: Desktop Data Visualization and Result Reporting**
 - Friendly UI dashboards for visualizing desktop test execution trends and pass/fail statistics.
 - Accurate reporting that automatically attaches full desktop or active window screenshots at the exact moment of failure.
 - Enable developers to quickly identify if failures stem from UI rendering issues, background process crashes, or backend data errors.

5.3.2 Non-Functional Requirements

- **Reliability:** Stable automation execution under continuous load, maintaining a persistent connection to the Windows desktop session without unexpected driver disconnects or framework crashes.
- **Performance:** Low-latency interaction with Windows UI elements and strictly optimized memory usage to prevent OutOfMemory (OOM) exceptions on the host machine during

prolonged test batches.

- **Scalability:** Easily extensible to test across multiple Windows operating systems (e.g., Windows 10, Windows 11) and adaptable to new underlying desktop technologies (such as WPF, WinForms, or UWP).
- **Usability:** Windows object repositories and automation scripts should be highly modular and reusable, with centralized configuration files that make it easy for QA teams to deploy and run tests across different local environments.

5.4 Feasibility Study

5.4.1 Contribution to Organisation Goals

Yes. IMSAxes Automation directly supports the organization's objective of minimizing manual QA effort while ensuring high reliability in our Windows desktop applications. It significantly accelerates regression testing cycles, reduces human error, and directly contributes to faster, more confident product release timelines.

5.4.2 Technical and Economic Feasibility

Yes. The system can be successfully engineered using reliable, currently available technologies such as Java, WinAppDriver (or UIAutomation), Maven, and Jenkins within acceptable timeline constraints. The overall economic implication is highly favorable, as it utilizes predominantly open-source tooling, with primary costs relegated strictly to execution infrastructure (e.g., Windows Virtual Machines) and initial development hours.

5.4.3 Integration Feasibility

Yes. The proposed Windows automation framework is inherently modular. It can be seamlessly integrated into existing CI/CD pipelines via Windows runner nodes, allowing tests to be triggered automatically upon new builds. Furthermore, its architecture can be easily adapted to support other internal Windows desktop projects or domains in the future.

5.5 New System Features

- Fully resilient Windows desktop automation execution, capable of recovering from unhandled application crashes and OS-level interruptions.
- Centralized, timestamped telemetry logging that captures every simulated Windows UI interaction (mouse/keyboard events) for all test scenarios.
- Support for unattended, batch-based execution of Windows test suites across distributed virtual machines.

- Interactive GUI-based dashboards providing pass/fail trend charts, granular filters, and active-window screenshots for highly traceable defect analysis.

5.6 Function of System

- **Admin Role:**

- Assign specific Windows automation modules (e.g., script creation or maintenance) to QA Engineers based on the project requirements.
- Review aggregate quality metrics and approve final test execution reports for release readiness.
- Manage employee profiles and monitor assigned task statuses.

- **Employee Role:**

- View, configure, and execute assigned Windows desktop test scripts locally or on remote nodes.
- Track the progress of executed test suites and utilize centralized logs and screenshots to debug individual script failures.
- Update the operational status of test cases (e.g., Passed, Failed, Needs Refactoring) and directly log defect root causes based on system feedback. Modify personal profiles and progress logs.

5.7 ACTIVITY DIAGRAM

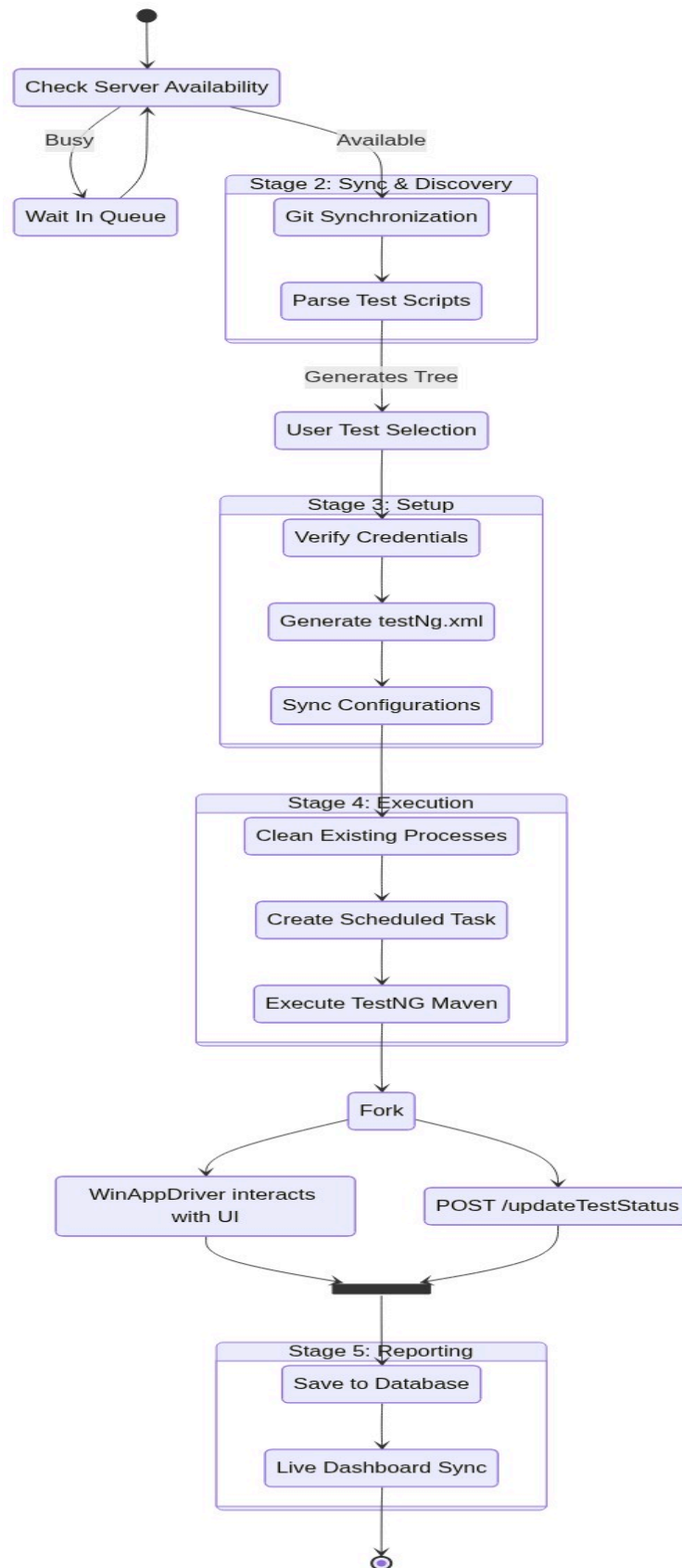


Fig. 5.1 – Activity Diagram for Automation Testing

5.8 USE CASE DIAGRAM

Depicts how different roles interact with the IMSAxes Automation Portal.

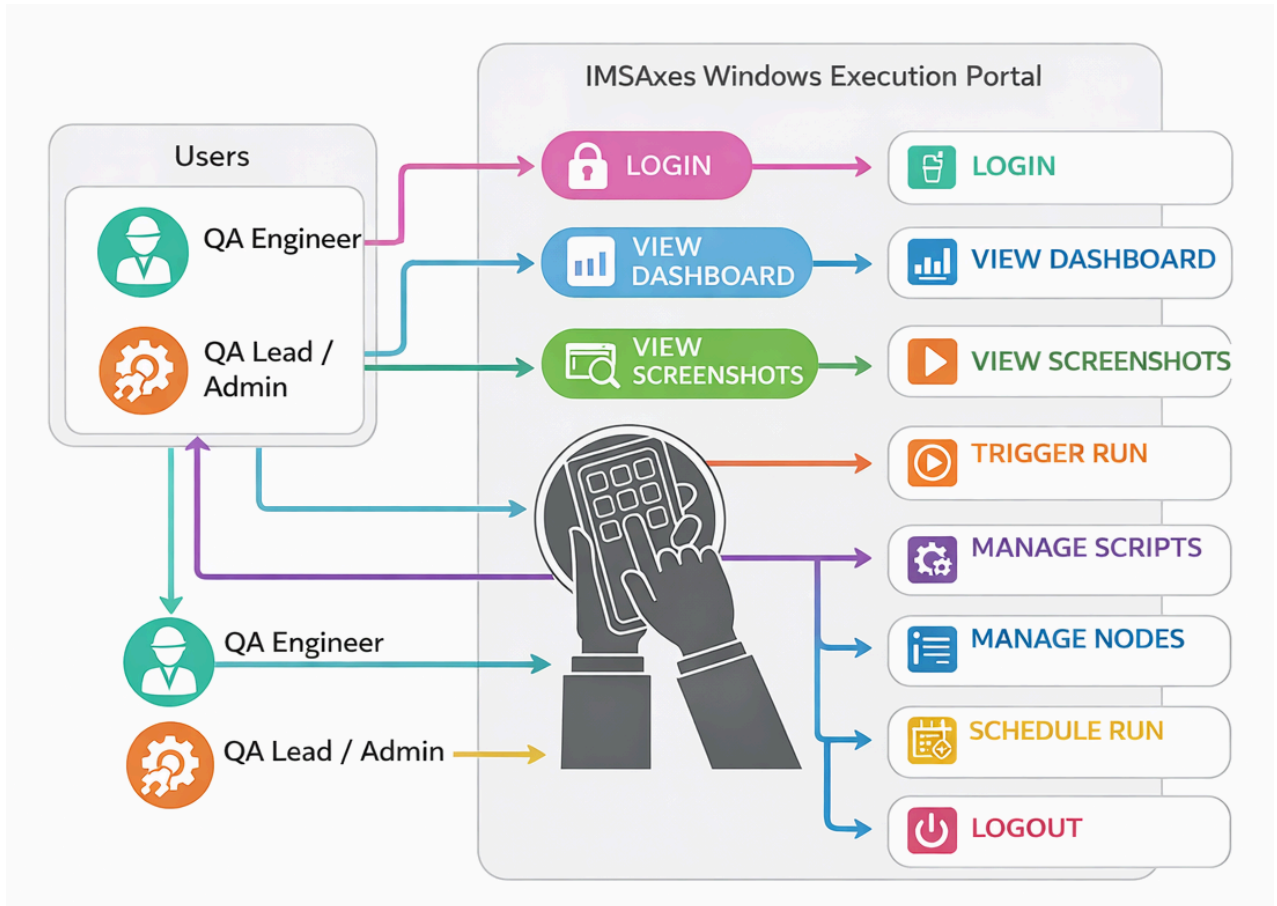


Fig. 5.2 – Use Case Diagram for IMAAxes Automation Portal

5.9 FRAMEWORK DIAGRAM

Outlines the layered architecture of the IMSAxes Automation testing framework.

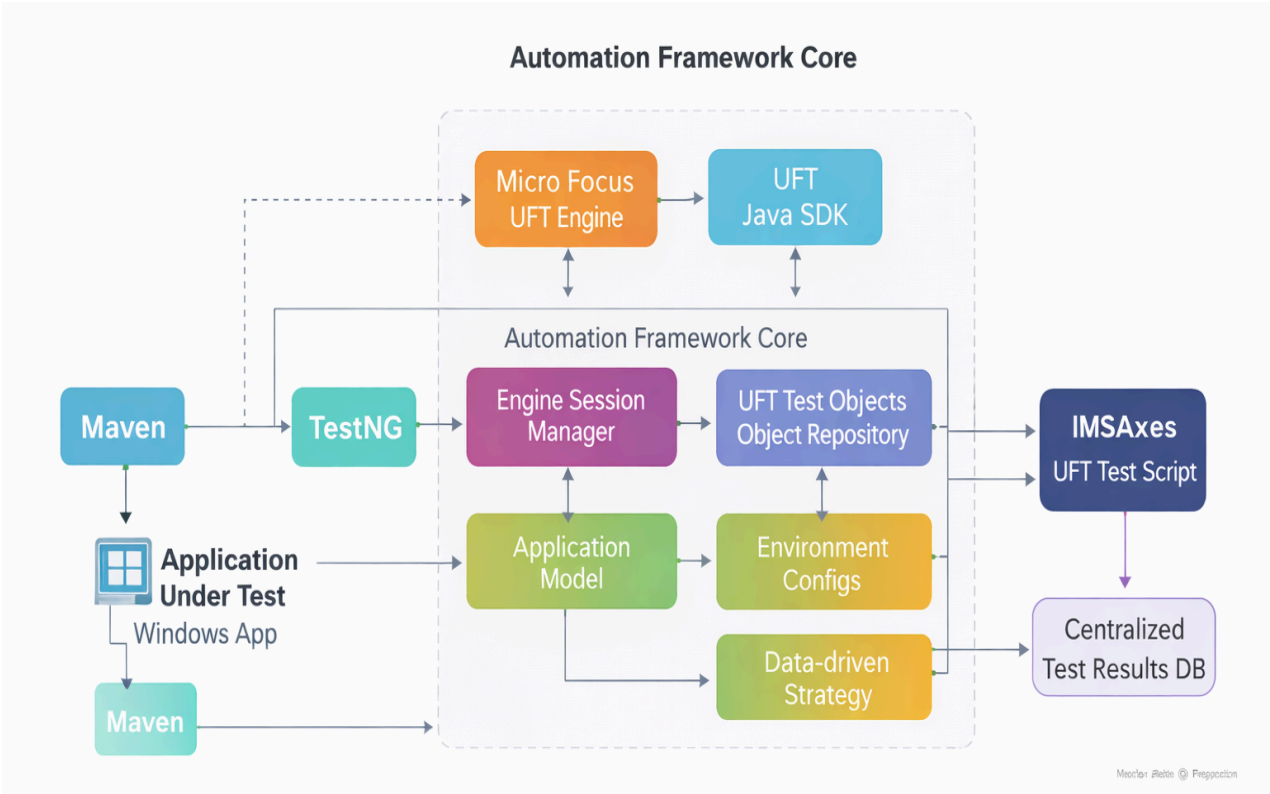


Fig. 5.3 – Framework for Automation Testing

5.10 SEQUENCE DIAGRAM

Describes how the automation testing flow operates step-by-step from test trigger to execution and result capture.

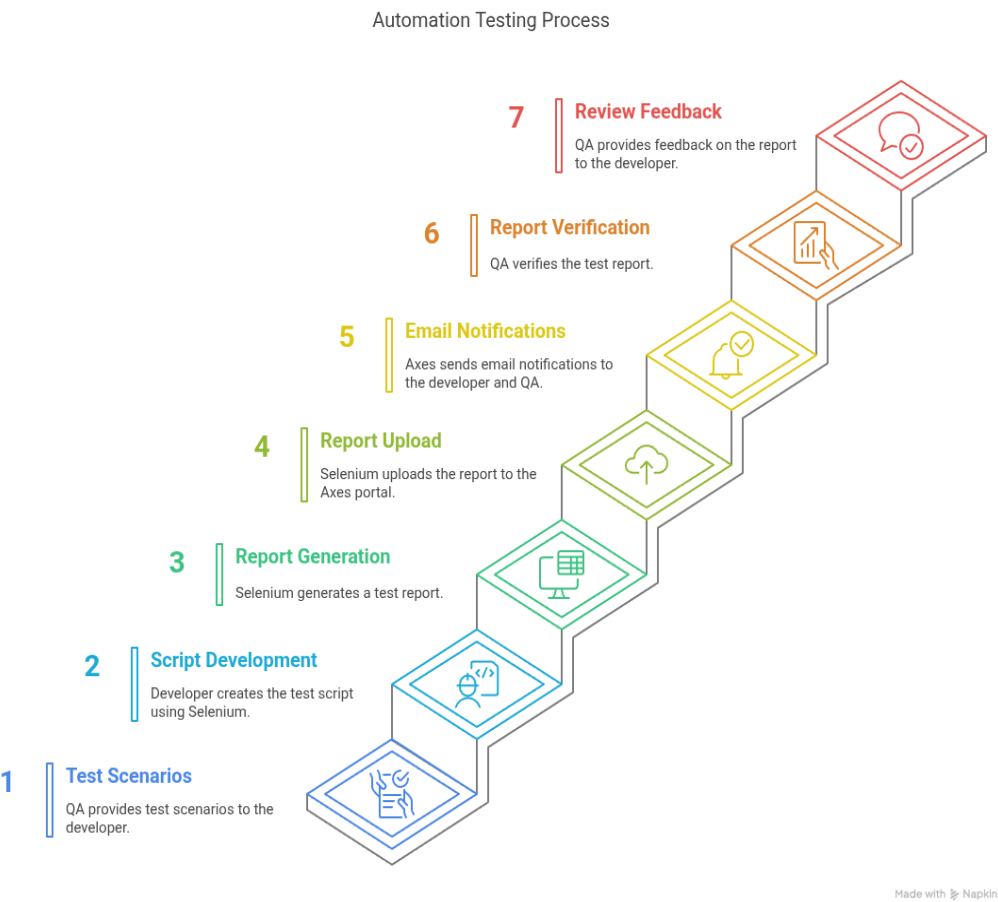


Fig. 5.4 – Sequence Diagram for Automation Testing

Chapter 6: IMSAxes Design

6.1 Login Page Design

The IMSAxes Automation Portal features a secure, React-based web interface designed for seamless interaction with the underlying remote Windows execution engines. The central login component and navigation routing are strictly optimized for QA Engineers and Automation Leads, enabling them to securely manage UFT test scripts, trigger remote execution batches, and monitor real-time telemetry without exposing the remote servers to unauthorized access.

The following figures represent the critical design validations and secure user interface interactions parameterized within the IMSAxes portal:

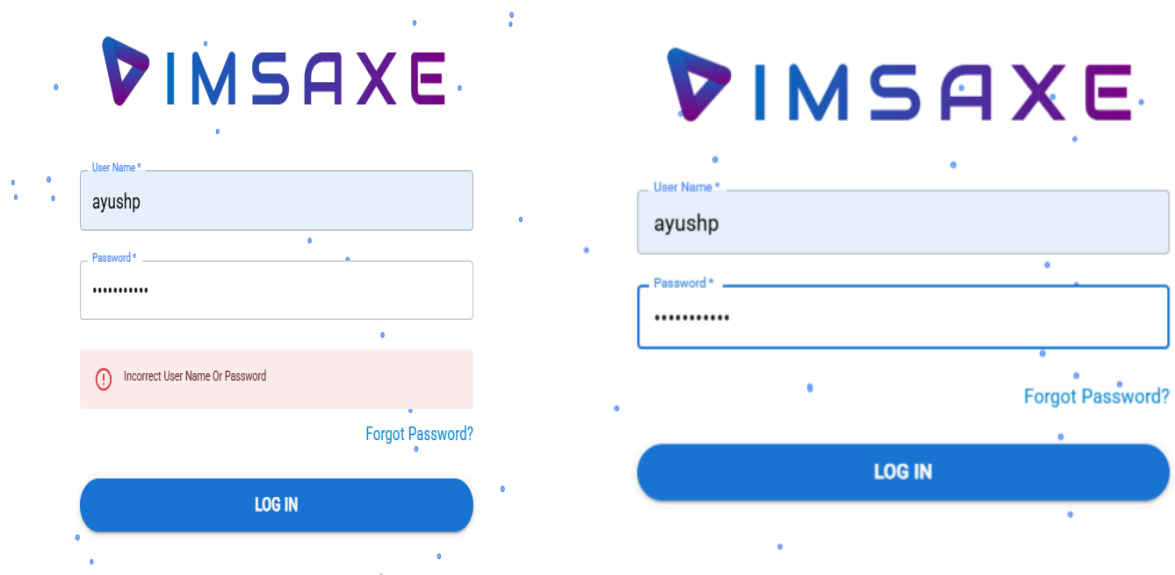
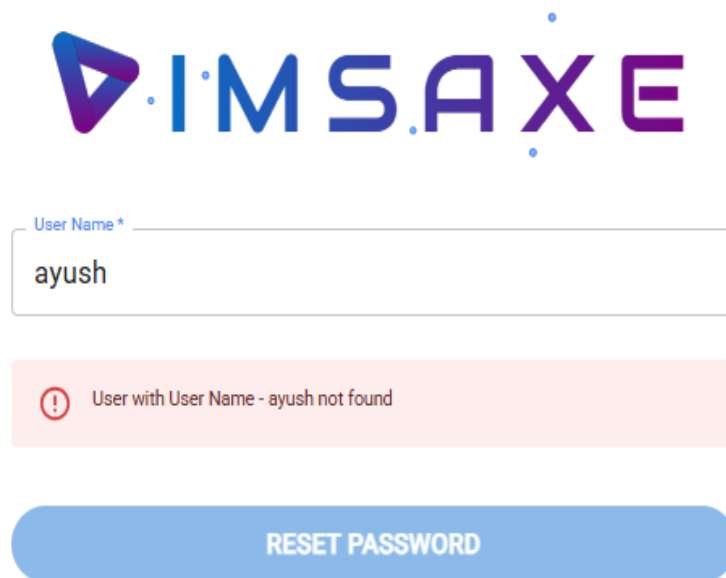


Fig 6.1 – Login Attempts

- Enter a username and an incorrect password.
- Click the "Login" button.
- Verify whether access is granted or an error message appears.



The screenshot shows the IMSAXE logo at the top. Below it, there are two main sections. The left section has a 'User Name*' field containing 'ayushp', a green success message 'Password reset email sent to email id ending in ****hp@meditab.com', and a 'RESET PASSWORD' button. The right section has a 'New Password*' field, an 'UPDATE PASSWORD' button, and a 'RESET PASSWORD' button at the bottom.

Fig 6.2 – Verify Forgot Password

The screenshot shows the IMSAXE logo at the top. Below it, there is a 'User Name*' field containing 'ayush'. Below the field is a red error message 'User with User Name - ayush not found'. At the bottom is a 'RESET PASSWORD' button.

Fig 6.3 – Verify Incorrect Credentials During Password Recovery

- Click on the "Forgot Password" link.
- Enter an incorrect or non-existent username.
- Validate whether the system shows an appropriate authentication error.

The image shows a web interface for updating a password. At the top, there is a logo for 'IMSA XE' with a stylized 'I' and 'M' in blue and purple. Below the logo is a text input field labeled 'New Password*'. Underneath the input field is a blue button with the text 'UPDATE PASSWORD'. Below this is a section for an email notification titled 'AXES password reset' with a small 'Inbox x' icon. The email is from 'axes@meditab.com' and is addressed 'to me'. The email content says 'Hi ayushp,' followed by 'We received a request to reset your AXES password. You can reset the password by clicking below button.' Below the text is a blue button labeled 'Reset Password'. At the bottom of the email section, it says 'This email is valid for 1 day.'

IMSA XE

New Password*

UPDATE PASSWORD

AXES password reset Inbox x

axes@meditab.com
to me ▼

Mon, Apr 13, 3:56 PM (20 hours ago)

Hi ayushp,

We received a request to reset your AXES password. You can reset the password by clicking below button.

Reset Password

This email is valid for 1 day.

Fig 6.4 – Update Password

- Enter the correct username.
- Click on the "Reset Password" button.
- Confirm that a password reset pop-up is triggered as expected.

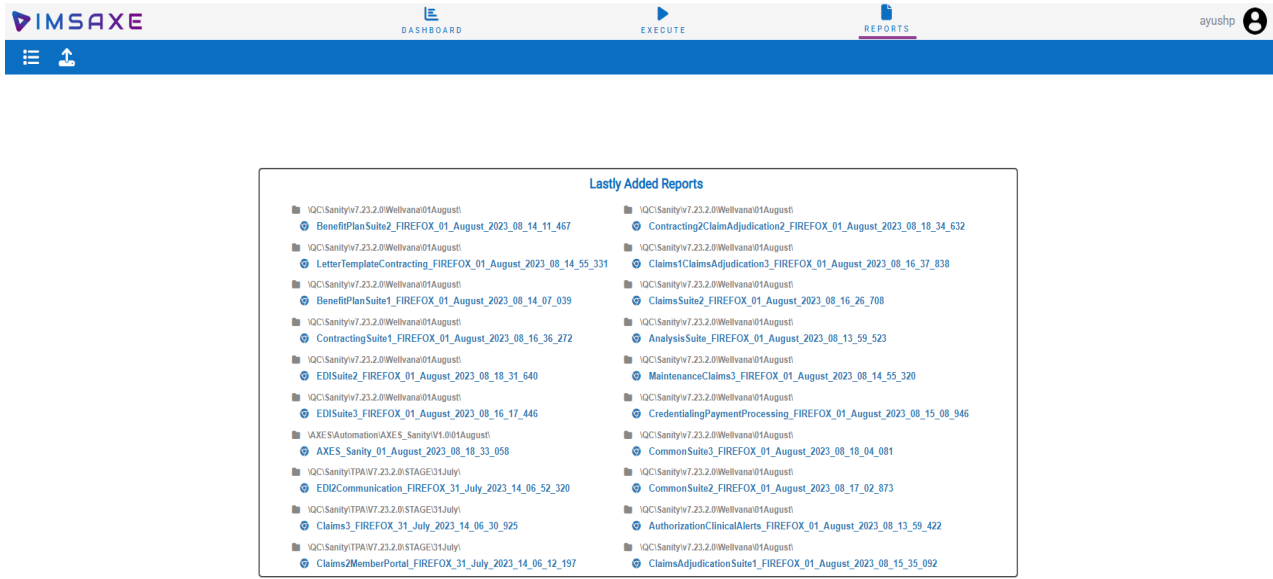


Fig 6.5 – Home Page on Successful Login

- Enter the correct username and password.
- Click "Login."
- Verify that the user is redirected to the dashboard or home page with available modules.

These interface checkpoints are important in ensuring that the user experience remains consistent and that all edge cases related to authentication are validated through automation scripts.

Screenshots and UI verification are typically performed during sanity and regression testing of the IMSAxes portal.

6.2 Interface Design

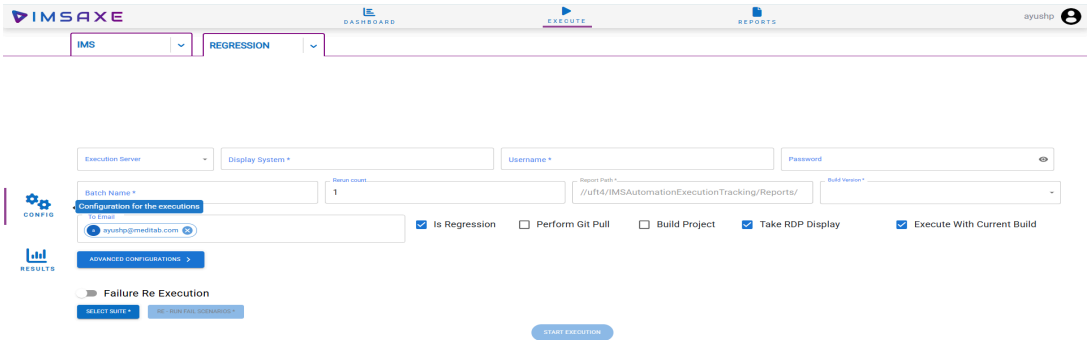


Fig 6.6 – Configuration Page

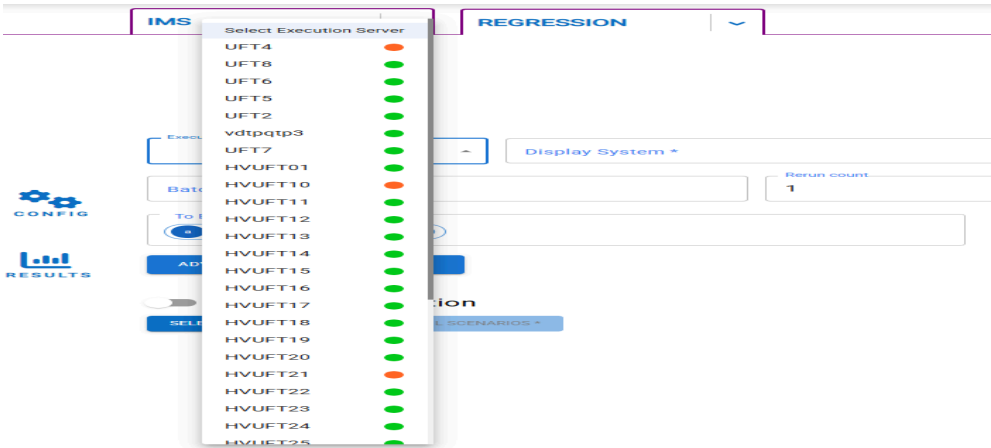


Fig 6.7 – Server Selection

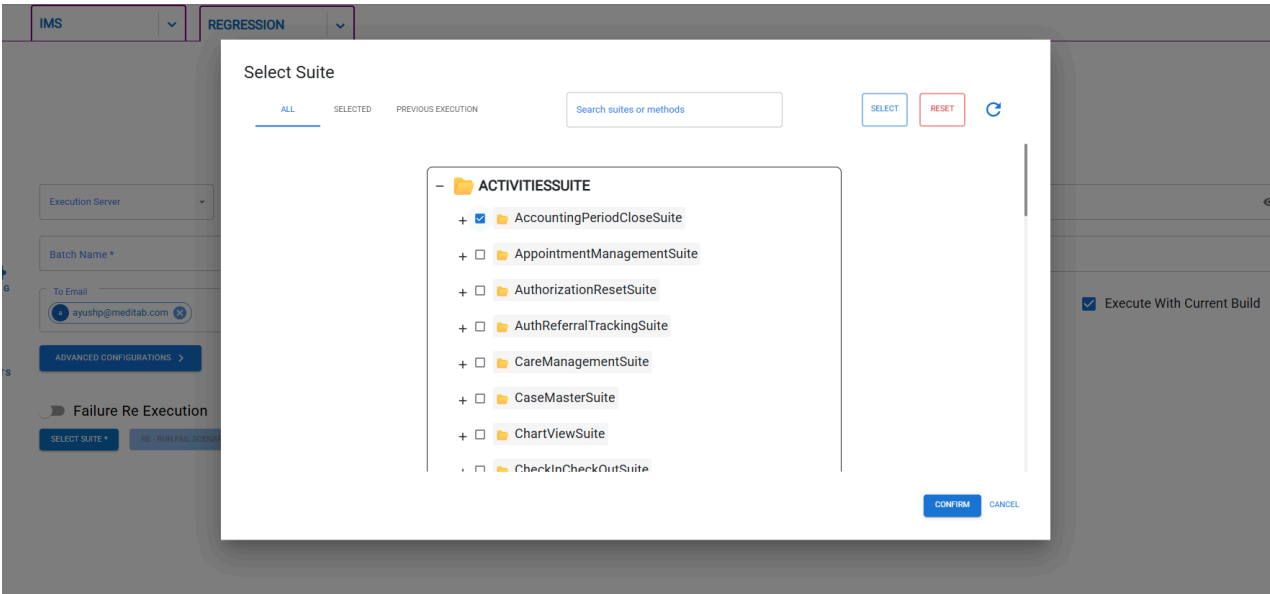


Fig 6.8 – Suite Selection

IMSAXE

Dashboard

EXECUTE

REPORTS

ayushp

QUICKCAP

SELECT...

Quick Fail Analysis

Jan 1, 2026 - Apr 13

Build

Own... (1)

Executor

BatchId

Suite

Fail Cat...

Config

Results

	B...	Batch	Suite	Test	DataProvider	RetryCount	Fail Category	Quick Failure Analysis
1.	3852	Re_MyTaskNoteDocFax_Suite	MyTaskNoteDocFaxSuite	verifyDocSignOffFromMyTaskAn...	0	2	Screen Interaction Issue	Failed to check enabled status of 'app_w_fra
2.	3852	Re_MyTaskNoteDocFax_Suite	MyTaskNoteDocFaxSuite	changeInFaxStatusVerification...	0	2	Data Issue	Failed to verify the actual record count and n
3.	3852	Re_MyTaskNoteDocFax_Suite	MyTaskNoteDocFaxSuite	checkCountsForAllCategoriesFr...	0	2	Image Issue	Failed to click on image with path '/Data/pag
4.	3852	Re_MyTaskNoteDocFax_Suite	MyTaskNoteDocFaxSuite	verifyEditFaxAndFiltersMyTask	0	2	Object Existence issue	Failed to check 'dw_doctor' DataWindow exis
5.	3852	Re_MyTaskNoteDocFax_Suite	MyTaskNoteDocFaxSuite	verifyAddNoteInDocumentFrom...	0	2	Crashing Issue	DB Crashing window popped-up
6.	3852	Re_MyTaskNoteDocFax_Suite	MyTaskNoteDocFaxSuite	verifyNotificationBarContMyTask	5	2	Data Issue	Failed to match Lab task count in Notification
7.	3852	Re_MyTaskNoteDocFax_Suite	MyTaskNoteDocFaxSuite	verifyNotificationBarContMyTask	4	2	Data Issue	Failed to match Reminder task count in Notifi
8.	3847	Re_DiagnosticLabOrderTracki...	DiagnosticLabOrderTrackin...	verifyTheDocCategoryChngInHL...	0	2	Dialog Interaction Issue	Failed to match dialog text. Expected: 'HL7 O
9.	3847	Re_DiagnosticLabOrderTracki...	DiagnosticLabOrderTrackin...	verifyOrderDateChangedWhenS...	0	2	Dialog Interaction Issue	Failed to match dialog text. Expected: 'HL7 O
10.	3847	Re_DiagnosticLabOrderTracki...	DiagnosticLabOrderTrackin...	verifyCollectedByInformationInH...	0	2	Data Issue	Failed to get Row Count from 'dw_master' Da
11.	3847	Re_DiagnosticLabOrderTracki...	DiagnosticLabOrderTrackin...	verifyLabTestShowLinkBasedOn...	0	2	Object Existence issue	Failed to get Row Count from 'dw_master' DataWindow.
12.	3847	Re_DiagnosticLabOrderTracki...	DiagnosticLabOrderTrackin...	checkGroupLabTestInOrderHisto...	0	2	Object Existence issue	Failed to verify the non existence of the test 4

1 - 100 / 714

Release | Settings | Search

Fig 6.9 – Report Page

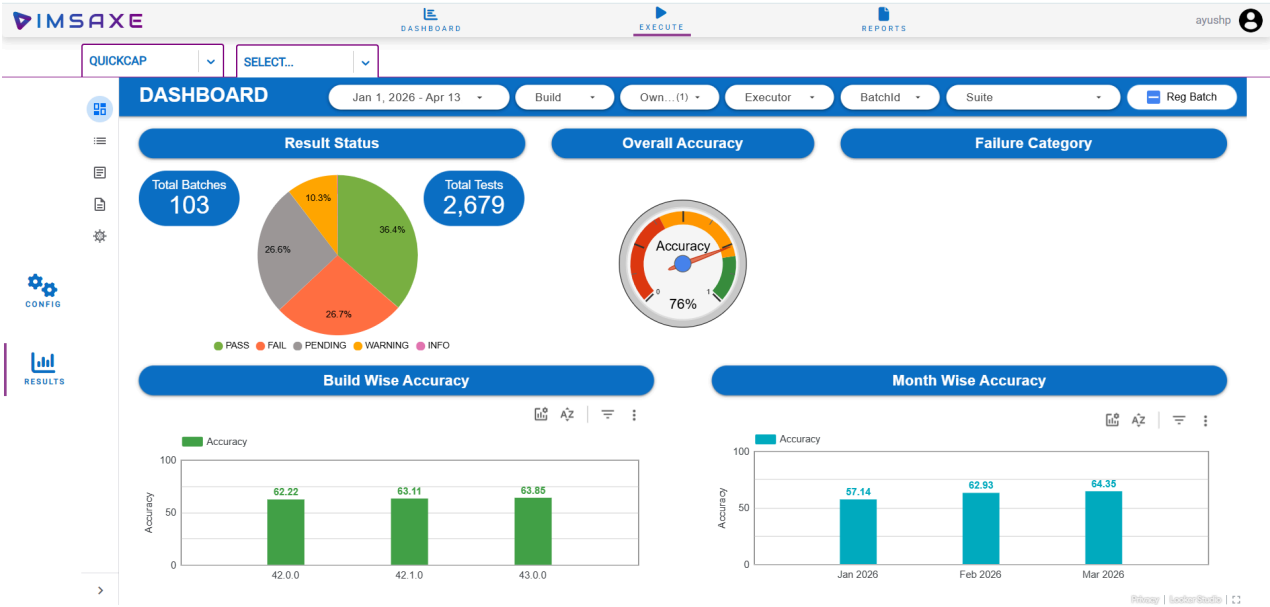


Fig 6.10 – Analysis Page

Chapter 7: Implementation Planning

7.1 Implementation Environment

For efficient and scalable Windows automation testing, a well-configured implementation architecture is crucial. The IMSAxes Automation framework relies on a decoupled, dynamic execution portal and a structured network of remote runner nodes to ensure smooth "lights-out" automation out-of-hours.

Environment Setup Requirements:

- **Automation Utility Interface:**
A centralized, React-based web dashboard (the IMSAxes Portal) is required to orchestrate execution. It enables QA Engineers to trigger multiple UFT execution batches across remote virtual machines without requiring manual Remote Desktop Protocol (RDP) access, drastically improving overall testing productivity.
- **Client Scenario Replication:**
The remote Windows machines (imsautomate02, etc.) must maintain cloned databases tailored to mimic the exact database size, schema, and registry configurations of a live clinical medical practice. This accurately replicates real-world usage of the IMS application and ensures realistic automation outcomes.
- **System Configuration:**
 - **High-performance hardware** is strictly required, as running the native Windows IMS.exe executable concurrently with the background Micro Focus UFT Engine consumes substantial system memory.
 - Intel Xeon E5 or equivalent, 16 GB RAM, 100 GB SSD storage.
 - The primary remote test machines must operate in isolated sessions natively during script execution to prevent OS-level focus loss or unhandled mouse-cursor interruptions.
- **Parallel Setup:**
 - To maintain execution continuity, dedicated Virtual Machines are segregated into strictly "Execution" and "Development" nodes. An additional system must be allocated solely for automation engineers to maintain and debug new Java/UFT scripts without interrupting live nightly regression suites.

This setup facilitates efficient test execution, reduces downtime, and improves resource utilisation in the testing lifecycle.

7.2 Program / Modules Specifications

As part of the internal IMSAxes project architecture, code was developed across both the web orchestration portal and the corresponding backend automation scripts. The core engineered modules include:

- **Secure Authentication Module:** Implementing JWT-based secure routing for QA personnel.
- **Environment & Test Discovery Module:** Node.js logic that parses remote Java TestNG repositories via regex to dynamically generate test execution trees.
- **Windows Process Manager:** PowerShell scripting integration that actively hunts and executes Taskkill on stale Windows processes (e.g., hanging ims.exe or uft.exe instances).
- **Real-time HTTP Telemetry:** Socket.IO and REST endpoints that consume live trace statements from the remote UFT execution and pipe them directly into the MySQL tracking database.
- **Dynamic Visual Dashboard:** Providing instant analysis on Pass/Fail metrics and providing direct hyperlinks to execution error screenshots.

Each architectural module was developed with strict micro-service principles and modular scalability in mind, enabling cross-client adaptability regardless of the specific test suites being executed.

7.3 Coding Standards

To maintain strict readability, reusability, and consistency across both the Node.js/React portal and the Java testing codebase, the following coding standards were strictly enforced:

- **Naming Conventions:**
 - Implemented universal CamelCase styling for variables and automation methods.
(checkUftEngineStatus(), triggerMavenJob())
- **Function Size:**
 - Methods were kept intentionally short and focused on a single logical responsibility (Single Responsibility Principle) to enhance testability and limit debugging scope.
- **Code Simplicity:**
 - Engineers adhered strictly to Agile principles, developing only what was architecturally necessary for current sprint requirements and avoiding convoluted premature optimization.

- **Reusability & encapsulation:**

- Frequently utilized UI operations were abstracted away from the business layer. In the context of UFT, Windows API interactions and static waits were encapsulated into a shared Application Model and global utility functions to prevent script duplication.

These unified standards effectively guaranteed a clean, extensible automation platform capable of scaling seamlessly alongside future testing requirements.

Chapter 8: Testing

8.1 Testing Plan

Testing is a critical phase in the automation project lifecycle, ensuring that the IMSAxes Automation framework consistently delivers reliable results across multiple QA scenarios, Windows environments, and execution nodes. The comprehensive testing plan for the IMSAxes project encompasses both validating the accuracy of the automation test results and ensuring the performance scalability of the underlying Node.js orchestration backend.

Automation Testing Plan

- **Automation Tools Required:**
 - Micro Focus UFT Engine (Unified Functional Testing) & UFT Java SDK.
 - TestNG Framework and Maven for dependency and lifecycle management.
 - PowerShell for OS-level execution orchestration.
 - React & Express (Node.js) for telemetry capture and centralized dashboard reporting.
 - Eclipse / IntelliJ IDE for Java script development.
- **Framework Characteristics:**
 - Modular and Reusable: Scripts are highly decoupled from business logic.
 - Application Model Architecture: Officially implements UFT's Application Model (similar to the Page Object Model), utilizing robust Object Repositories instead of brittle dynamic locators.
 - Data-Driven and Batch Support: Supports massive parallel execution clusters and dynamic data injection from external SQL/Excel environments.
 - Real-Time Telemetry: Bypasses traditional static HTML reports by streaming execution data over HTTP callbacks directly into an interactive React dashboard.
- **Scope Definition:**
 - **In-Scope:**
 - Functional testing of the target native IMS Windows application workflows (Login, Patient Registration, etc.).
 - Validation of the standalone IMSAxes Web Portal (Task management,

trigger buttons, reporting modules).

- Cross-OS execution testing (testing the framework against Windows 10 vs. Windows Server deployments).
- Automated nightly build Regression and Sanity checks.

- **Out-of-Scope:**

- Non-UI backend database structural validations.
- Deep security penetration testing of the corporate network.

- **Test Suite Preparation:**

- Structured, hierarchical test suites (testNg.xml) are generated dynamically by the backend.
- Data sets are parameterized securely using centralized configuration files deployed via PowerShell.
- Test suites are rigidly organized into predefined clusters for scheduled unattended nightly runs vs. specific on-demand interactive runs.

- **Scripting and Execution Schedules:**

- Weekly UFT script reviews and Object Repository updates based on newly released IMS.exe application features.
- Daily remote execution for continuous regression and sanity validation.

- **Automation Testing Deliverables:**

- Robust UFT/Java automation test scripts.
- Modular Windows UI utility functions and Object Repositories.
- Live centralized execution telemetry traces stored in MySQL.
- A fully interactive React summary dashboard for upper-management stakeholder visibility.

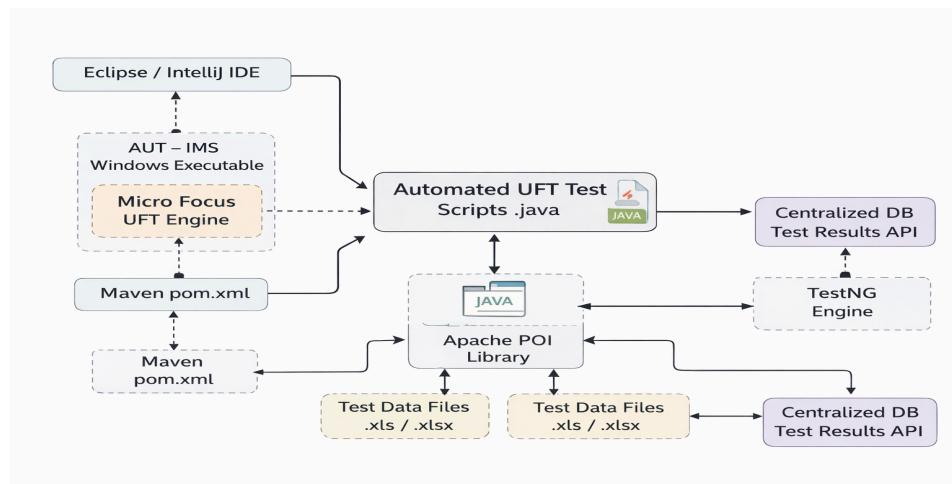


Fig. 8.1 – Automation Testing Flow Chart

Performance Testing Plan

The performance testing strategy was designed to ensure that the newly built IMSAxes web portal and its backend Node.js architecture could scale efficiently without dropping crucial telemetry data during massive parallel execution batches. Key performance metrics include API response times, WebSocket throughput handling, and Database read/write latency.

- **Load Testing:**
 - Simulate high-volume HTTP callback traffic (simulating over 1,000 parallel test executions posting results simultaneously) to verify that the Express backend processes the payload and returns a status 200 OK in under 2 seconds.
- **Stress Testing:**
 - Gradually increase WebSocket connections (simulating an excessive number of attached remote runner nodes) to determine the threshold before the Node server crashes or begins dropping live broadcast events.
- **Network Simulation:**
 - Test the React frontend and VNC proxy responsiveness in artificially throttled, low-bandwidth intranet scenarios.
- **Database Load Testing:**
 - Execute massive continuous asynchronous INSERT operations (e.g., simulating 500 test cases failing and dumping massive stack traces simultaneously) to monitor latency limits in the MySQL database.
- **System Resource Monitoring:**
 - Measurably track Windows CPU spikes and memory consumption footprints on the remote testing VMs when running ims.exe concurrently alongside the uft.exe engine.
- **Scalability and Stability:**
 - Verify the response time and overall stability of the dashboard under low, moderate, and absolute peak testing load conditions (e.g., during end-of-month regression sweeps).

8.2 TESTING STRATEGY

For Automation Testing

The strategic approach focuses intensely on identifying high-ROI (Return on Investment) test cases that are the strongest candidates for desktop automation:

- Tests cases that must be identically repeated across multiple weekly internal application builds.
- Workflows that are mathematically complex and highly susceptible to human fat-finger errors during manual data entry.
- Clinical scenarios requiring significantly large dynamic datasets (e.g., mass billing simulations).
- Core application workflows (Smoke tests) representing high-risk business functionalities.
- Complex desktop verification cases that are simply infeasible for manual QA to complete within the 48-hour release window.
- Time-consuming manual procedures that tie up critical QA resources unnecessarily.

This overarching strategy guarantees that IMS releases are vetted consistently, rapidly, and natively at an enterprise scale with every subsequent patch cycle.

Chapter 9 – Conclusion

9.1 Problems Encountered and Possible Solutions

For Performance and Backend Scalability:

Encountered Problem: A critical performance bottleneck was observed during the initial development of the telemetry logging system. When massive parallel test suites executed simultaneously, the high volume of incoming HTTP Post payload broadcasts caused the Node.js event loop to block, leading to delayed dashboard updates and database timeout errors.

Implemented Solution: Following a deep architectural investigation, the backend payload processing was restructured asynchronously. Real-time updates were offloaded exclusively to lightweight Socket.IO channels, and complex MySQL insert queries were refactored into optimized, bulk-batched INSERT operations to drastically reduce connection overhead.

For Automation Testing Connectivity:

Encountered Problem: Early iterations of the Windows runtime execution suffered from "ghost executions," where stale uft.exe or ims.exe background processes from previous failed runs prevented new automation batches from launching correctly, requiring tedious manual intervention via Remote Desktop.

Implemented Solution: A dynamic PowerShell process-hunting mechanism (start_execution.ps1) was engineered to systematically aggressively execute a Taskkill against any orphaned target processes prior to triggering the Maven jobs, ensuring a highly sterile execution environment.

For Script Architectural Maintenance:

Encountered Problem: Legacy automation scripts contained significant redundancy relying heavily on primitive WinApp locator strategies, causing severe maintenance overhead whenever the target Windows application updated its UI framework.

Implemented Solution: The codebase was heavily abstracted to strictly utilize the UFT Application Model (Object Repository). This fundamentally detached business logic from

Chapter 9 – Conclusion

object identification, ensuring that a single UI update only required a single tweak within the repository rather than hundreds of script edits.

9.2 Summary of Internship Work

The performance testing component taught us the importance of scalable application design and proactive load simulation. Testing under various user loads and response times helped us verify system integrity under stress conditions.

- **Saved Critical Testing Time:** Achieved by enabling unattended, parallel batch executions across remote virtual machines.
- **Reduced Manual Errors:** Eliminated the risks associated with manual environment configuration and human data-logging.
- **Increased Team Productivity:** Provided QA engineers with a centralized, "single pane of glass" dashboard to effortlessly monitor global test results.
- **Ensured Consistent Testing:** Locked automation executions into a strict, version-controlled repository to guarantee precision across all future software builds.

The project demonstrated that both automation and performance testing are crucial for modern, client-facing applications.

Chapter 10 – Limitations and Future Enhancements

10.1 Limitations

- **Cannot Replace Exploratory Intelligence:** Automation scripts execute strictly defined paths. They cannot replace the human intuition required for exploratory testing, nor can they accurately evaluate subjective qualities like front-end user experience (UX) and aesthetic design.
- **Susceptibility to Windows UI Volatility:** Despite utilizing robust UFT Object Repositories, significant architectural changes to the underlying Windows .exe forms often result in stale object references. This requires diligent and regular maintenance of the automation codebase.
- **100% Automation is Infeasible:** Designing scripts for highly complex, edge-case medical scenarios often yields diminishing returns. Achieving 100% automation coverage is practically infeasible due to the sheer cost, time, and escalating architectural complexity required to maintain it.

10.2 Future Enhancements

- **AI-Assisted Test Generation:** Developing a machine-learning utility capable of automatically scaffolding boilerplate UFT automation scripts and Object Repository mappings directly from plain-text BDD (Behavior-Driven Development) test case inputs.
- **Predictive Data Generation:** Implementing dynamic, AI-based mock data generators that automatically seed the required SQL databases with randomized, anonymized medical profiles prior to script execution.
- **Automated Performance CI/CD Integration:** Expanding the current functional pipeline to integrate backend load-testing dynamically, automatically halting deployments if the Node.js API performance degrades beyond acceptable network metrics.
- **Predictive Dashboard Analytics:** Enhancing the current React dashboard with AI anomaly detection, allowing the UI to flag irregular execution times or recurring failure trends proactively, rather than simply displaying raw pass/fail statistics.

Chapter 11 – Bibliography/ References

For reference, I used the following websites for help:

(<https://adnhelp.microfocus.com/leanft/>)

(<https://react.dev/>)

(<https://socket.io/docs/v4/>)

(<https://nodejs.org/en/docs/>)

(<https://testng.org/doc/>)

(<https://maven.apache.org/>)

(<https://mvnrepository.com/>)

(https://www.tutorialspoint.com/software_testing_dictionary/performance_testing.html)

Chapter 12 – Appendix

